

KMP 字符串匹配

$S[N]$ $P[M]$
↓ ↓
目标串 模板串

暴力匹配: for (int i=1; i<=n; i++)

bool flag = true;

for (int j=1; j<=m; j++)

if ($S[i+j-1] \neq P[j]$)

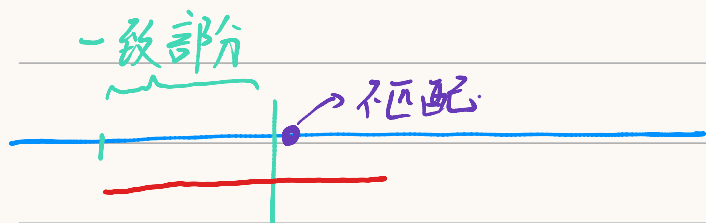
flag = false; break;

对于原串 S 中的每个字符, 都尝试以其为起点, 判断之后的字符是否能与模板串 P 匹配上

字符串都从 1 开始

$O(mn)$

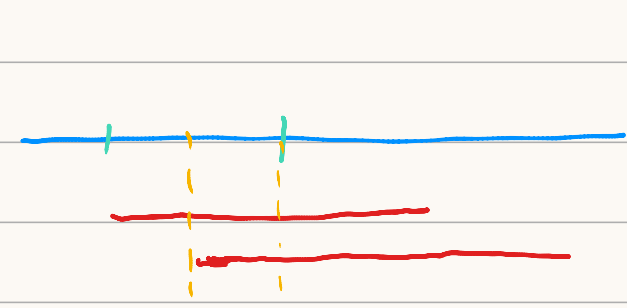
KMP 之前匹配即使不成功, 我们也已经有了信息, 利用其优化



暴力做法是把模板串向右移动 1 格, 重新匹配

我们可以利用一致部分中的信息, 让模板串多走几格

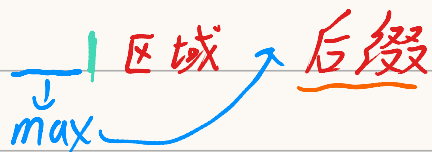
我们要找的, 就是 ~~模板串~~ 模板串最少能走几步,



↓ 移动完之后, 使这一段与目标串匹配上

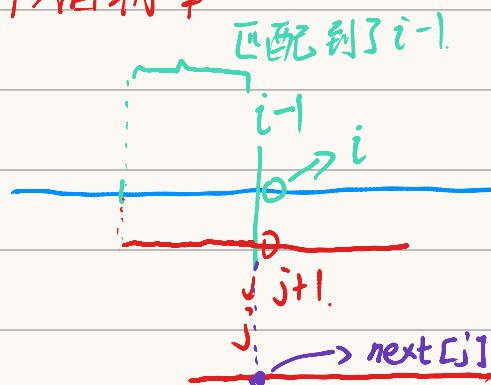
这样模板串就不用再重新从第1位开始匹配

我们要找出已判断的目标串
与模板串前缀最大重合部分
最长公共前后缀



$next[i]$ 的含义: $p[1, j] = p[i-j+1, i]$

P 为目标串

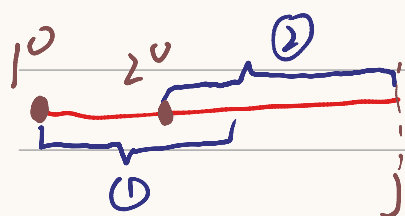


$i-1$ 与 j 匹配上了 $S[i-1] = P[j]$

但 i 与 $j+1$ 不匹配 $S[i] \neq P[j+1]$

j 走到 $next[j]$, 再判断

$S[i]$ 与 $P[next[j]+1]$ 是否匹配



② = ① 所以 2° 走到 1° 之后,
前面那段相同部分就不用再动了

如果不匹配, 那就从 $next[j]$ 到 $next[next[j]]$

例:

$P = "a b a b a b a b"$

$S = "a b a b a b c"$

1 2 3 4 5 6 7 8

最大前后缀 1

$next[1] = 0$

$next[2] = 0$

$next[3] = 1$

a b a

$next[4] = 2$

a b a b

$max = 2$

$next[5] = 3$

a b a b a

$next[6] = 4$

a b a b a b

$next[7] = 5$

a b a b a b a

$next[8] = 6$

a b a b a b a b

1 2 3 4 5 6 7
 S a b a b a b c i 此时 $S[i] \neq P[j+1]$
 P a b a b a b a b $j = \text{next}[6] = 4$
 $\therefore j$ 跳到 4

a b a b 中的内容一定是重合的

因为 a b 与 a b 的内容一一对应，而 a b a b
 是最长公共前后缀

所以现在继续比较 $S[i]$ 与 $P[\text{next}[j] + 1]$
 $O(m+n)$

831. KMP字符串

题目

提交记录

讨论

题解

视频讲解

给定一个字符串 S ，以及一个模式串 P ，所有字符串中只包含大小写英文字母以及阿拉伯数字。

模式串 P 在字符串 S 中多次作为子串出现。

求出模式串 P 在字符串 S 中所有出现的位置的起始下标。

输入格式

第一行输入整数 N ，表示字符串 P 的长度。

第二行输入字符串 P 。

第三行输入整数 M ，表示字符串 S 的长度。

第四行输入字符串 S 。

输出格式

共一行，输出所有出现位置的起始下标（下标从 0 开始计数），整数之间用空格隔开。

数据范围

$1 \leq N \leq 10^5$

$1 \leq M \leq 10^6$

输入样例：

```
3
aba
5
ababa
```

输出样例：

```
0 2
```

难度：

简单

时/空限制：

1s / 256MB

总通过数：

151266

总尝试数：

295989

来源：

模板题

算法标签

```

1 #include <stdio.h>
2
3
4 const int N = 1e5 + 10; const int M = 1e6 + 10;
5 int n, m;
6 char p[N], s[M];
7
8 int nxt[N];
9
10 int main() {
11     // 字符串从1开始
12     scanf("%d", &n); scanf("%s", p + 1);
13     scanf("%d", &m); scanf("%s", s + 1);
14
15     // 2 求解nxt数组的过程
16     // 只需要在模式串中处理, i从2开始, 因为P[1] = 0;
17     // 意味着模式串第一个字符不匹配, 那么退无可退, 下一次
18     // 仍然要从1开始匹配
19     // i是先锋, 最终i的结果满足
20     // P[1, j] == P[i - j + 1]
21     for(int i = 2, j = 0; i <= n; i++) {
22         while(j && p[i] != p[j + 1]) {
23             j = nxt[j];
24         }
25         if(p[i] == p[j + 1]) j++;
26         nxt[i] = j;
27     }
28     // 我们可以借鉴1的过程理解2
29     // 实际就是P的前段与后段进行字符串匹配
30
31
32
33     // 1 KMP的匹配过程
34     // 我们是S[i]与P[j + 1]匹配, 所以j从0开始
35     // 这里是为了简化操作, j从0开始, j=0就代表已经到头了
36     // 下一步i与j+1匹配, 对应两个串是i和1匹配, 从头开始
37     // 0相当于一个哨兵, 提示我们的模式串要从头开始了
38     for(int i = 1, j = 0; i <= m; i++) {
39         while(j && s[i] != p[j + 1]) {
40             j = nxt[j];
41         } // 循环结束有两种可能
42         // 1 j退到开头0, 不处理, 等i++
43         // 2 S[i] P[j + 1]匹配, i++, j也要++
44         if(s[i] == p[j + 1]) j++;
45         if(j == n) {
46             // 匹配成功
47             printf("%d ", i - n);
48             // 因为最终结果要从0开始, 所以 - 1变为i - n
49             j = nxt[j];
50             // 匹配完成后, j也要回退一次, 继续匹配
51         }
52     }
53
54     return 0;
55 }

```